CME295 Lecture 1 - Transformers

Lecture plan

1 Introduction
2 Tokenization
3 Word representation
4 Self-Attention mechanism
5 Transformer architecture
6 End-to-end example

1 INTRODUCTION

Transformer architecture appeared in 2017 and was ***super scalable*** compared to what community had before, like the more data and compute you feed into transformer the more outcome you get.

That's how it looks like

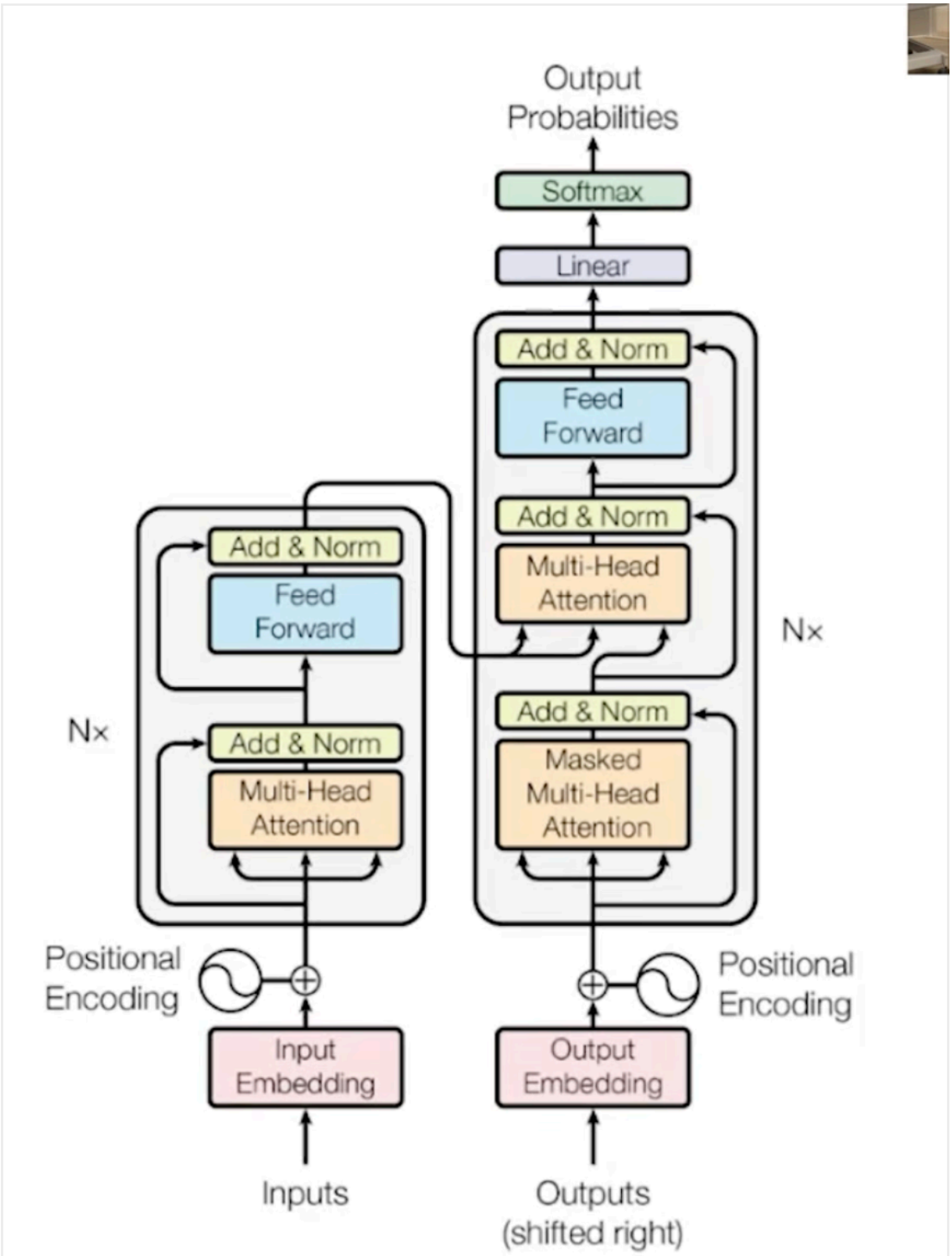


Figure 1: The Transformer - model architecture.

Stanford

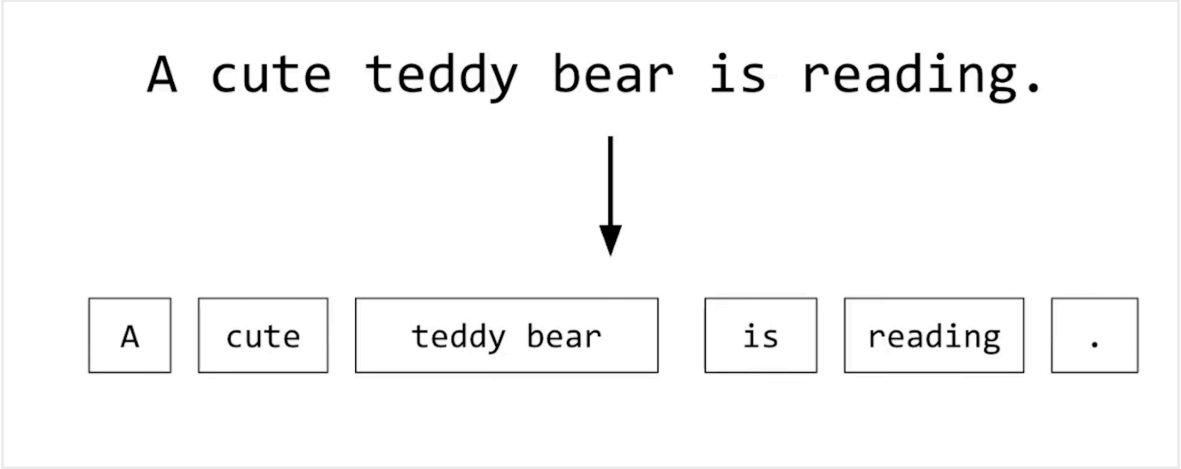
In November 2022 chatgpt was introduced, chatgpt is LLM, LLMs are based on transformer architecture.
Today we have agent systems like Claude Code, Codex, etc.

2 TOKENIZATION PART

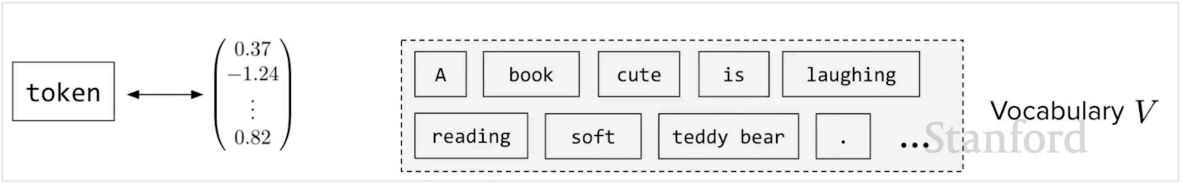
Models don’t understand text or characters, they understand numbers. That’s

we need to somehow represent our text in numbers

Here we split our example sentence into indivisible block like «A», «cute», «teddy bear» and so on
Each block is called a ***token***

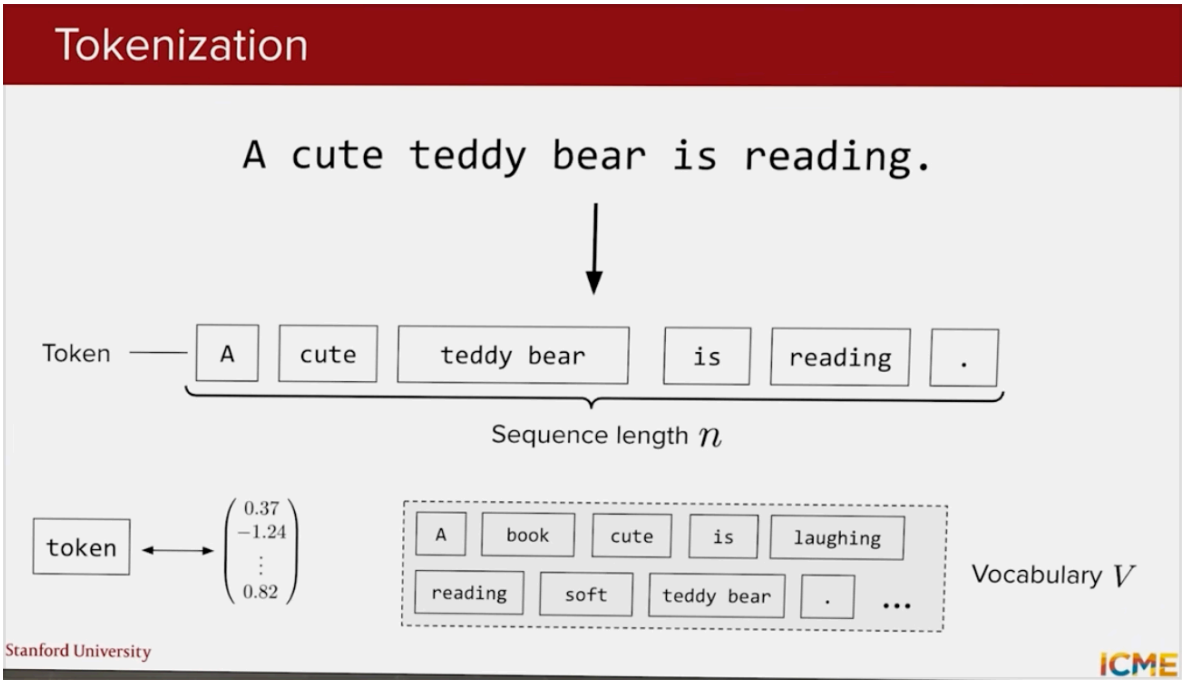


We want to represent each *token* mathematically in a form of numbers ***vector***

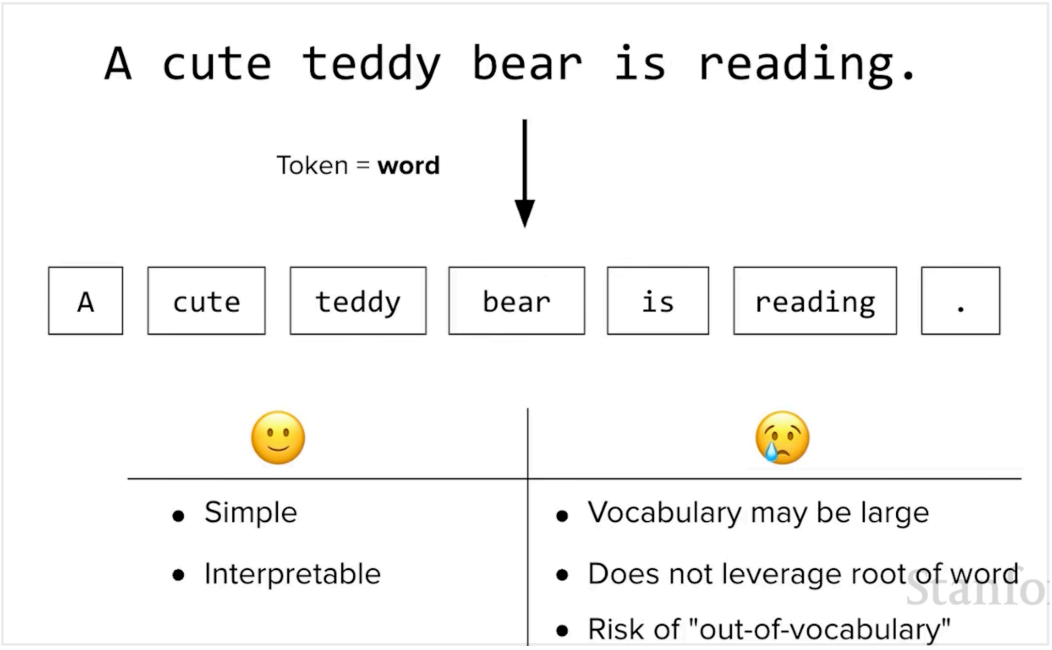


Vocabulary encapsulates all possible tokens presented in your text.
Different vocabulary choices ~ different sequences length

6 tokens in our example

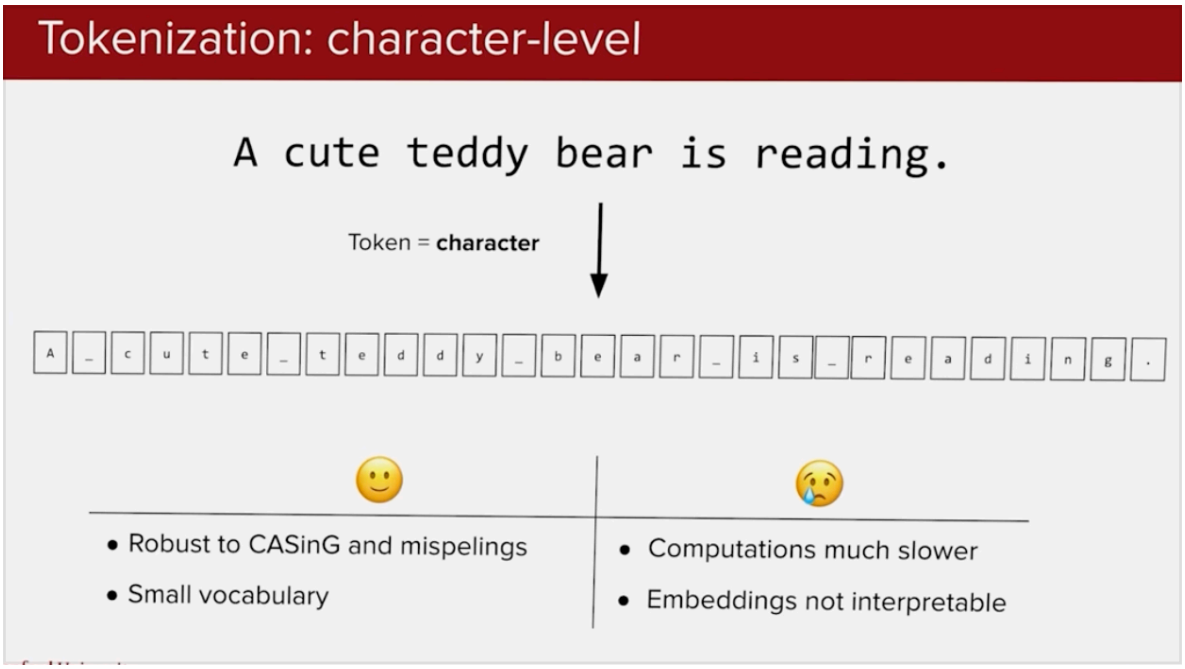


Lets use ***spaces as delimiters***, there are pros and cons represented in the figure below



With such an approach we don't pay much attention to ***root of the word***, that's a ***main problem***.
Thus vocabulary becomes large.
For example, two independent tokens for «bear» and «bears»
We want to have different behaviour

Character-level tokenization



Pros:
1 now we are robust to misspelling and unknown words because we are taking into account all vocabulary
2 our vocabulary size is equal to alphabet size

Cons:
1 computations is much slower because our ***model is a function of sequence***

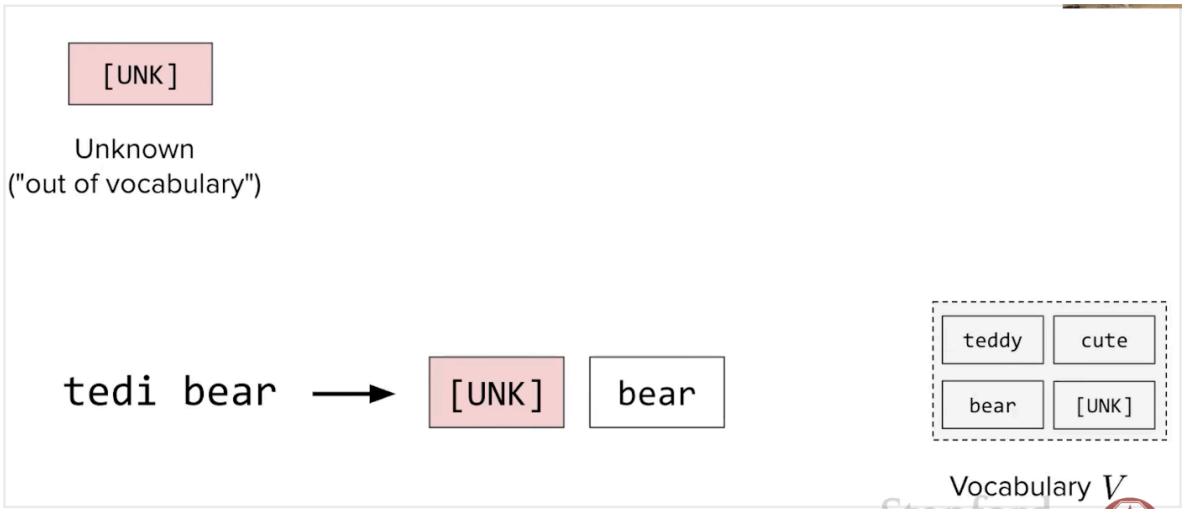
length, so it takes more time to process a whole sequence
2 embedding become not interpretable like what if we have two embedding for orange and apple, they may have been closer to each other, thus allowing to interpret them somehow

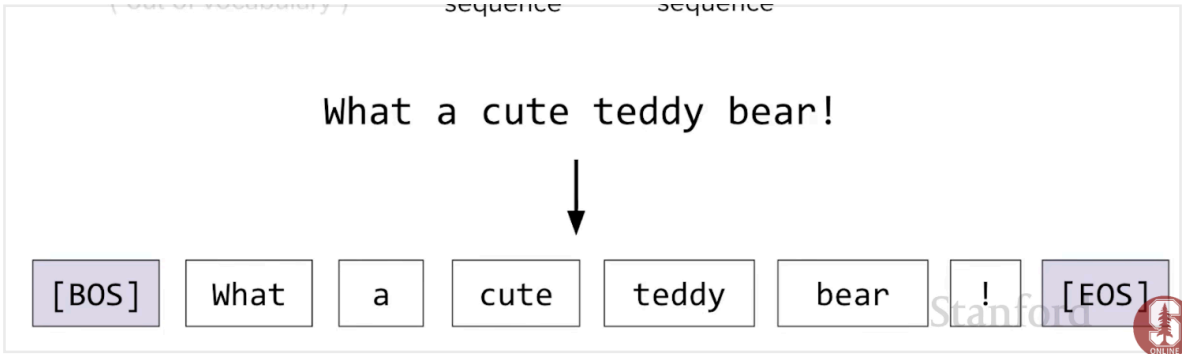
LETS TRY SOMETHING IN BETWEEN
Using subwords may be the best choice, we will use roots of words. They will be learned from the data
Like for bear and bears we will extract subword bear

You will hear a lot Acronym ***BPE*** (byte per encoding) - type of subword tokenizer. BPE learns what tokens should be in your vocabulary
Principle: starts from initial set of tokens and identifying the most frequent pairs in your training data and comes out with dedicated tokens to merge tokens of these pairs

Example: starts from every letter in eng alphabet and then merges like «a» and «n» getting «an» token

Special tokens





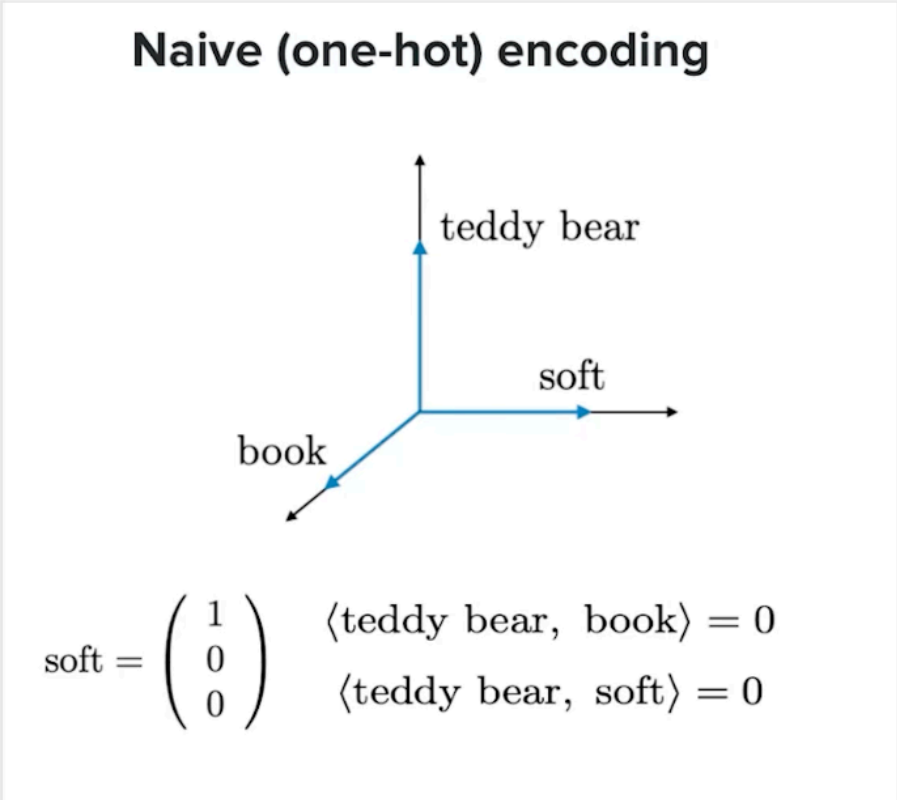
1 UNK - for unknown words in text
2 BOS - start the generation process (feed this token into your model)
3 EOS - the end of the sequence to tell that generation is finished
4 PAD - padding token (to make your sequences be of a certain length)
Note: It is not universal way to define tokens

Note: for ***today's LLMs*** the ***size of vocabulary*** is equal to approximately ***200,000*** and it handles mostly English but also other languages tokens. Most common is BPE tokenizer for defining tokens

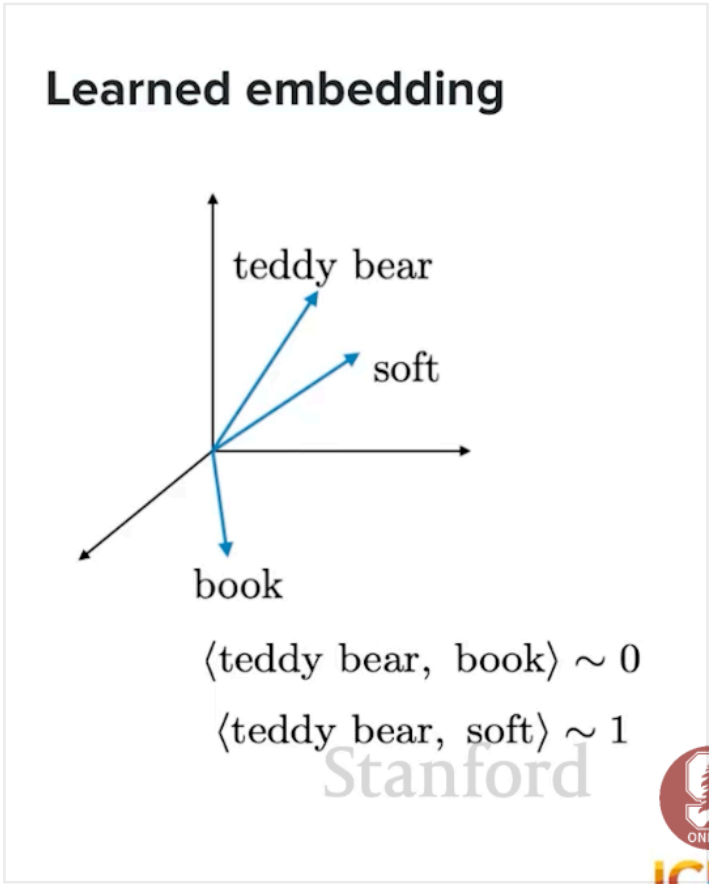
3 WORD REPRESENTATION

There are two main approaches of representing tokens

3.1 One hot encoding

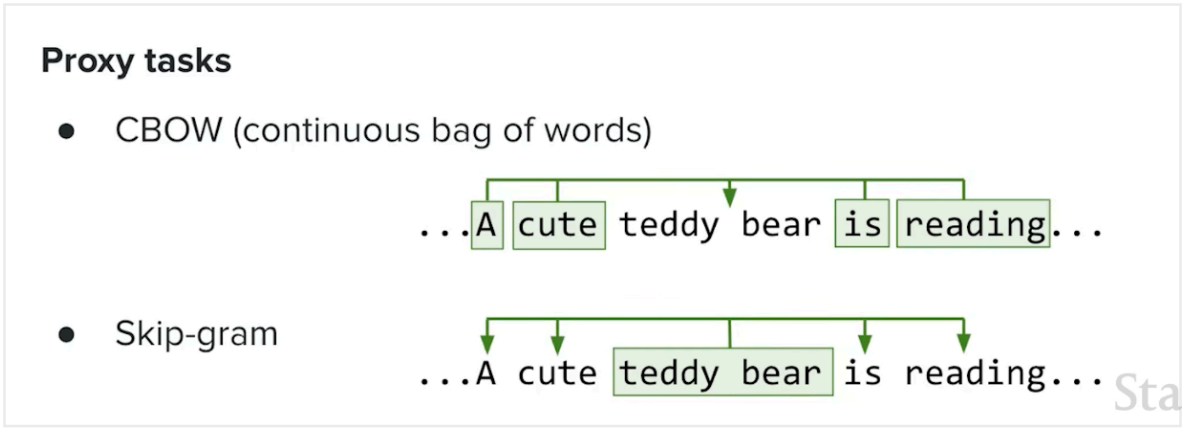


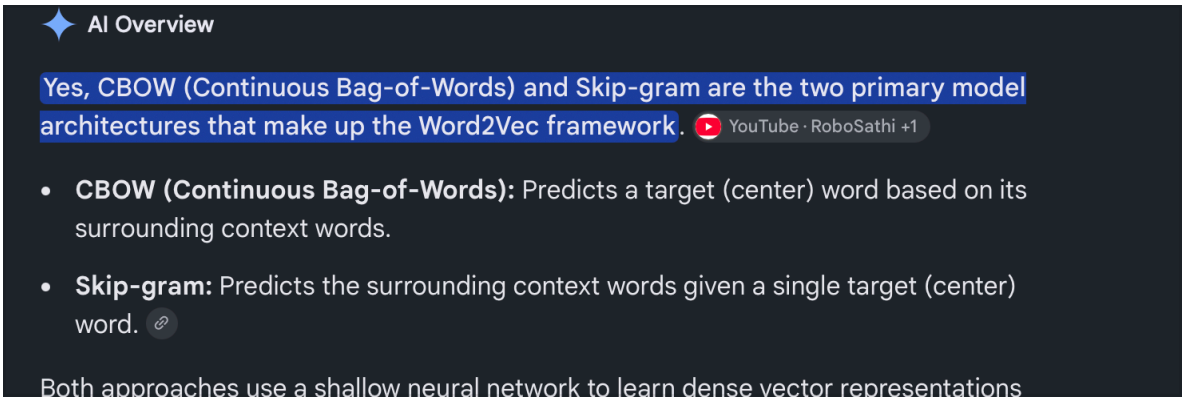
2 ***Learn*** tokens representations in a way to see how they ***relate to each other***



For tokens representations (embeddings) learning we apply technique called ***proxy task*** learning that stands for learning task not as primary goal but learning it for something else

WORD2VEC is an algorithm for learning tokens representation, two approaches are CBOW and Skip-grams

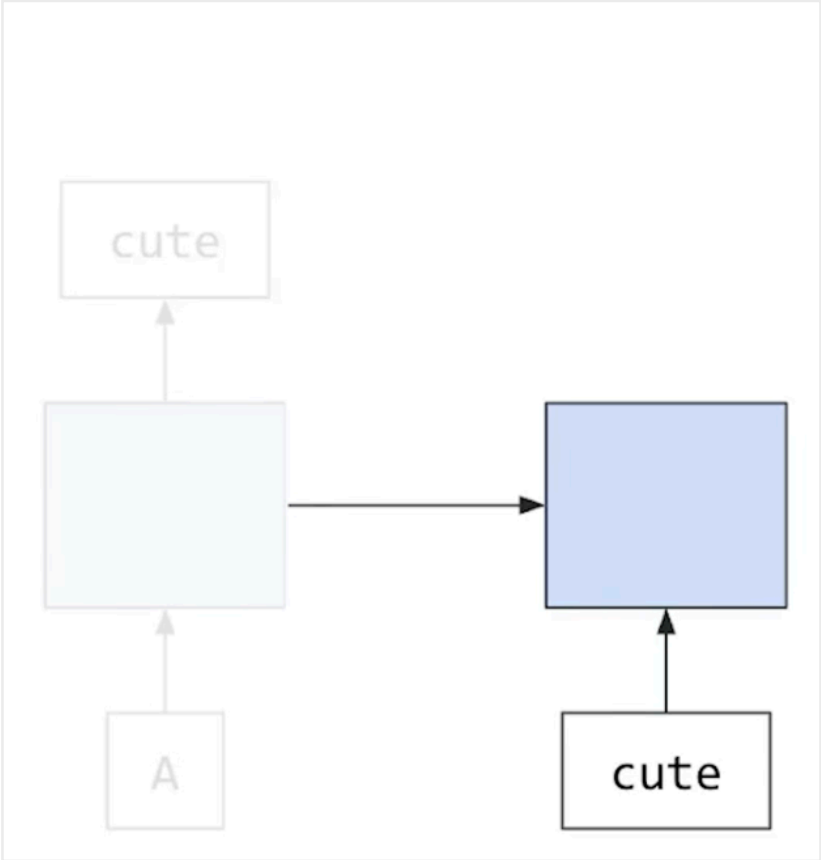




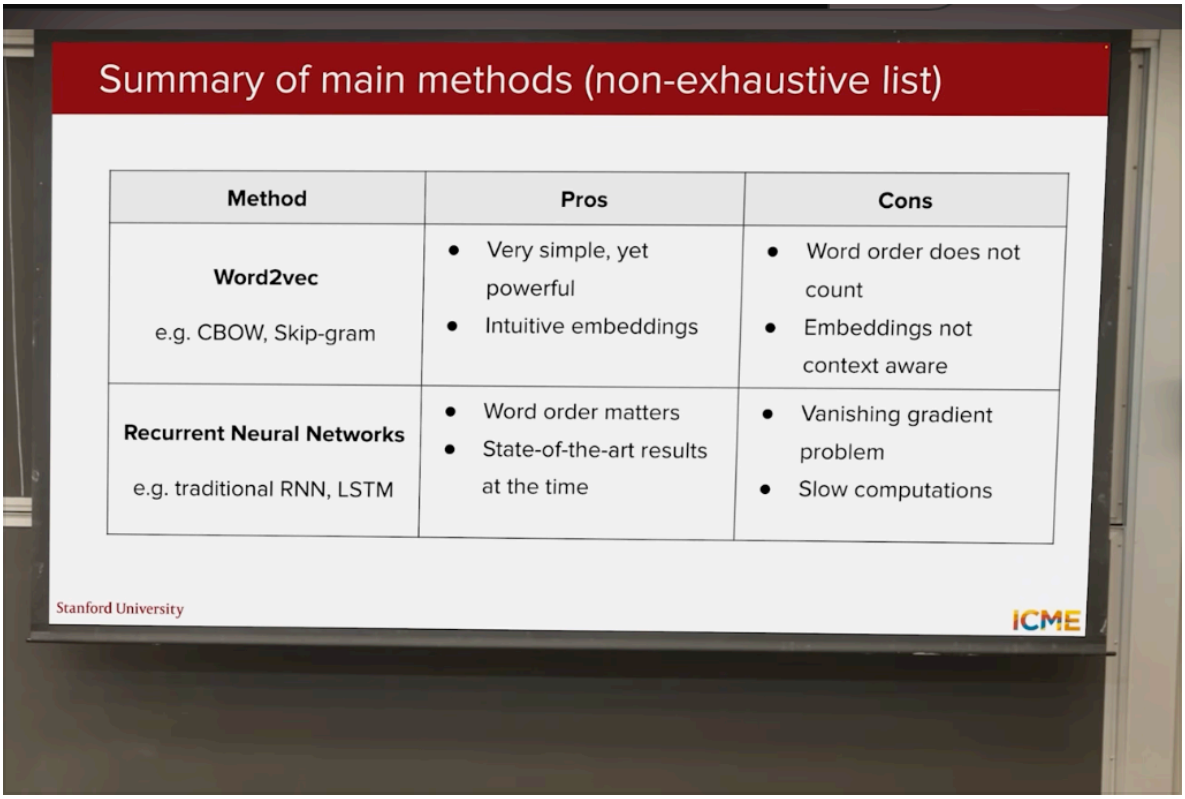
Limitations
1 The main problem of this approach is the representation of one chosen word for example bank of the river and going to the bank will have the same representation.
2 The order doesn't matter
Like teddy bear hugs baby child and baby child hugs teddy bear (the same embeddings for each sentence)

To solve this ***problem*** people used RNNs (dominated 2010s)
extra quantity called ***hidden state*** was computed, which encapsulated the meaning of sentence which it saw so far, ***passed*** it through one block of RNN to another

Thus it allowed you taking into consideration the order of the words in the sentence



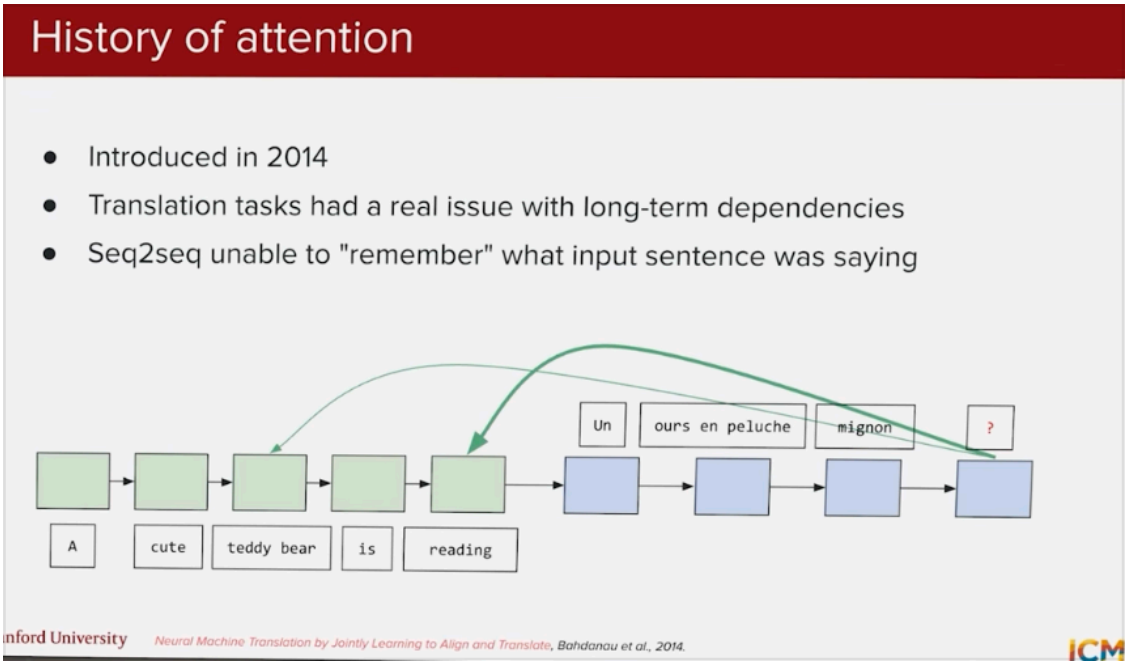
Picture to recap



Attention mechanism arises in 2014

Instead of relying solely on hidden state mechanism it offered another idea

The idea: to let your model to have strict connections from tokens that you have seen in the past



4 SELF-ATTENTION MECHANISM

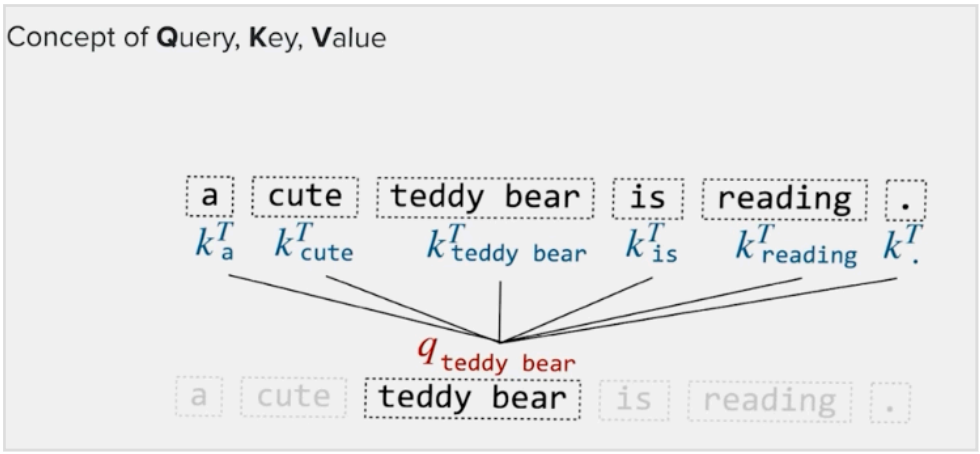
SELF-ATTENTION is ***attention*** but on steroids

Key principle: instead of computing the representation of current token with respect to hidden state that encapsulates the meaning of a sequence we've seen so far including this word, we calculate the representation ***as a function of all words in sequence***
Note: You may think that you lose words order, but we will address this moment later

Notations
QKV, Q - query, K - key, V - values

1 Q, Query - is the entity that tries to determine what this token is about
It wants to find tokens that are useful to construct a ***meaningful representation*** of the token that it's responsible for

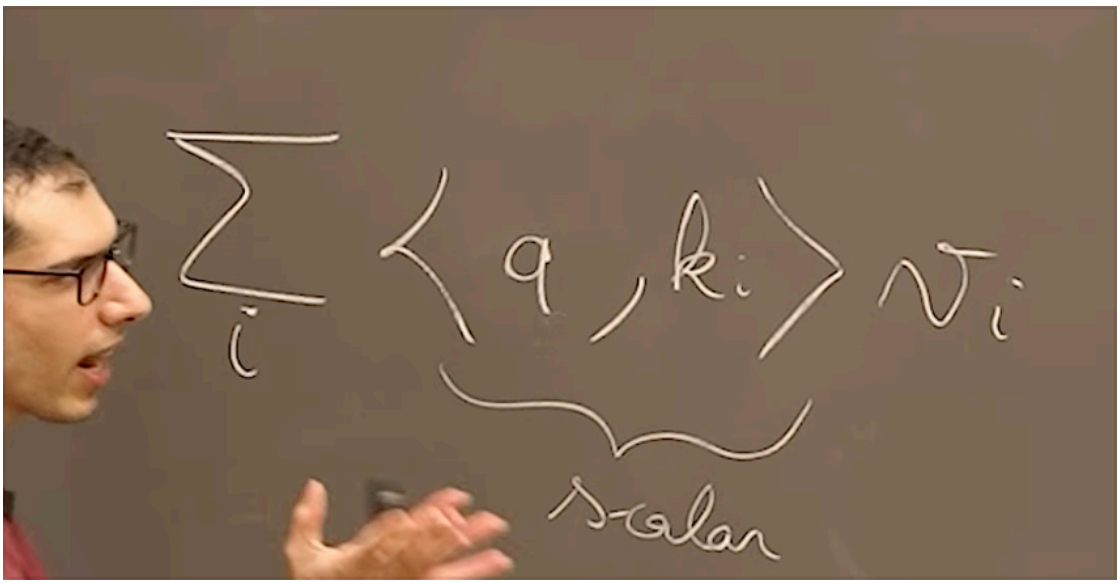
It will check how similar this quantity to keys of the sequence, key is associated to each token



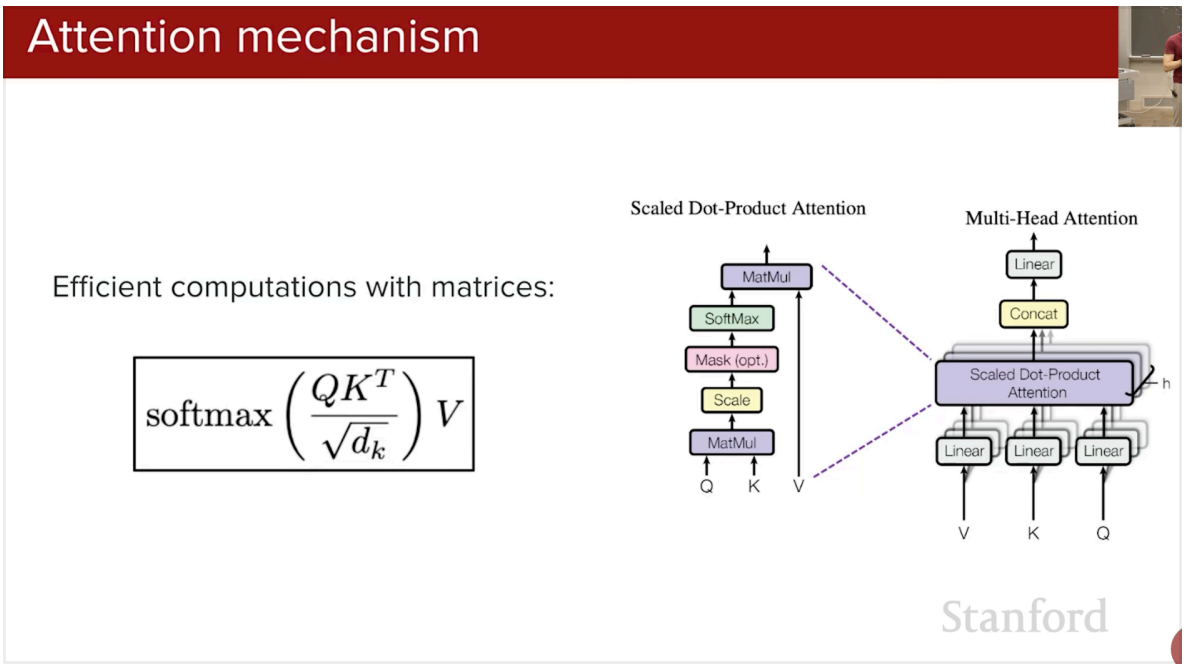
2 K or key, associated to each token
3 V or value, associate a value to each token after seeing how similar each key to query for some word W (W=teddy bear)

In other words, we want to represent current token (q for «teddy bear») as a function of all other tokens in a sequence
So we will determine weights to see how similar each taken to q for «teddy bear»
And use these weights to compute actual value, calculations below

Example: if q for «teddy bear» and k for «cute» will be large, than calculation will heavily depend on this amount



Q,K,V - projection matrices
Input token multiplied using them to have a meaningful representation of a token



5 Transformer architecture

We are ***not*** doing ***hidden state passing*** anymore
instead every token interacts with every other one through direct connections (self-attention)
that's also why paper called as «Attention is all you need»

Encoder on the left - ***computes meaningful representations or embeddings*** for input text fed as input
Decoder on the right - ***use representations from encoder along with some internal embeddings*** in order ***to generate translation token by token***
Transformer - is a foundation for modern LLMs

1:12:20 - walk through each component of transformer
ENCODER
Before passing input text into encoder we have to process it
1 split input text into tokens
2 for each token - it's static representation or embedding, static means embedding is not aware of context; model will just know what word it is
3 add positional embedding for each token representation, thus making model aware of words position

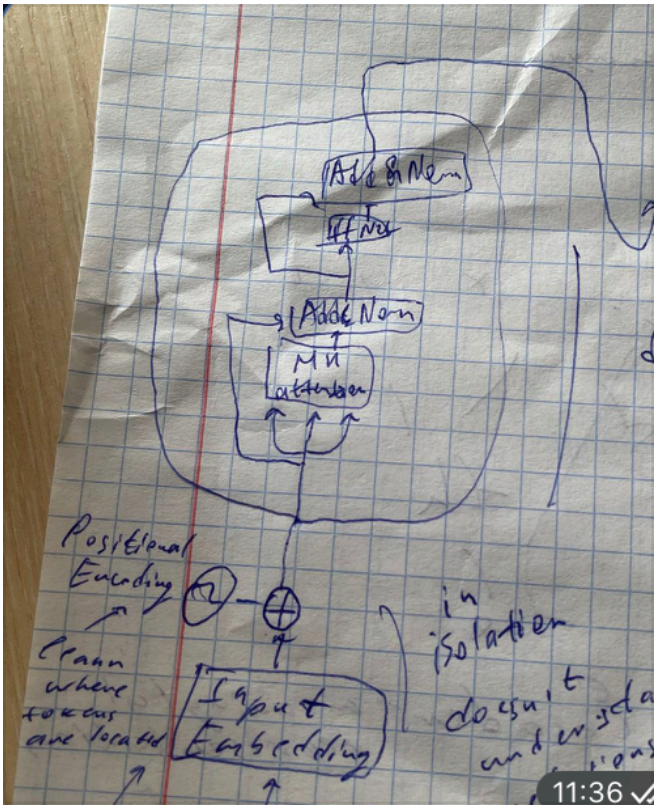
- Positional embedding may be learned or precomputed relying on frequencies

4 these mixed embeddings (input embeddings + positional) are passed into encoder

- for each token they were computed in isolation, meaning they don't represent relationship in text; model will just know words meaning and their order
  in sentences «bank of the river» and «robbing the bank» word «bank» will have the same embedding (just different position)

In encoder part we process static embeddings getting context-aware dynamic embeddings at the top of encoder

- Encoder block uses projection matrices Q, K, V that allow to represents tokens as a function of different tokens making them context-aware, dynamically computed
- After encoder block, in sentences «bank of the river» and «robbing the bank» word «bank» will have different embeddings, in first one «bank» embedding will be aware of information about «river», in the 2nd case «bank» will be aware of «robbing», «thief», etc.



DECODER

Decoder has two MH Attention layers.

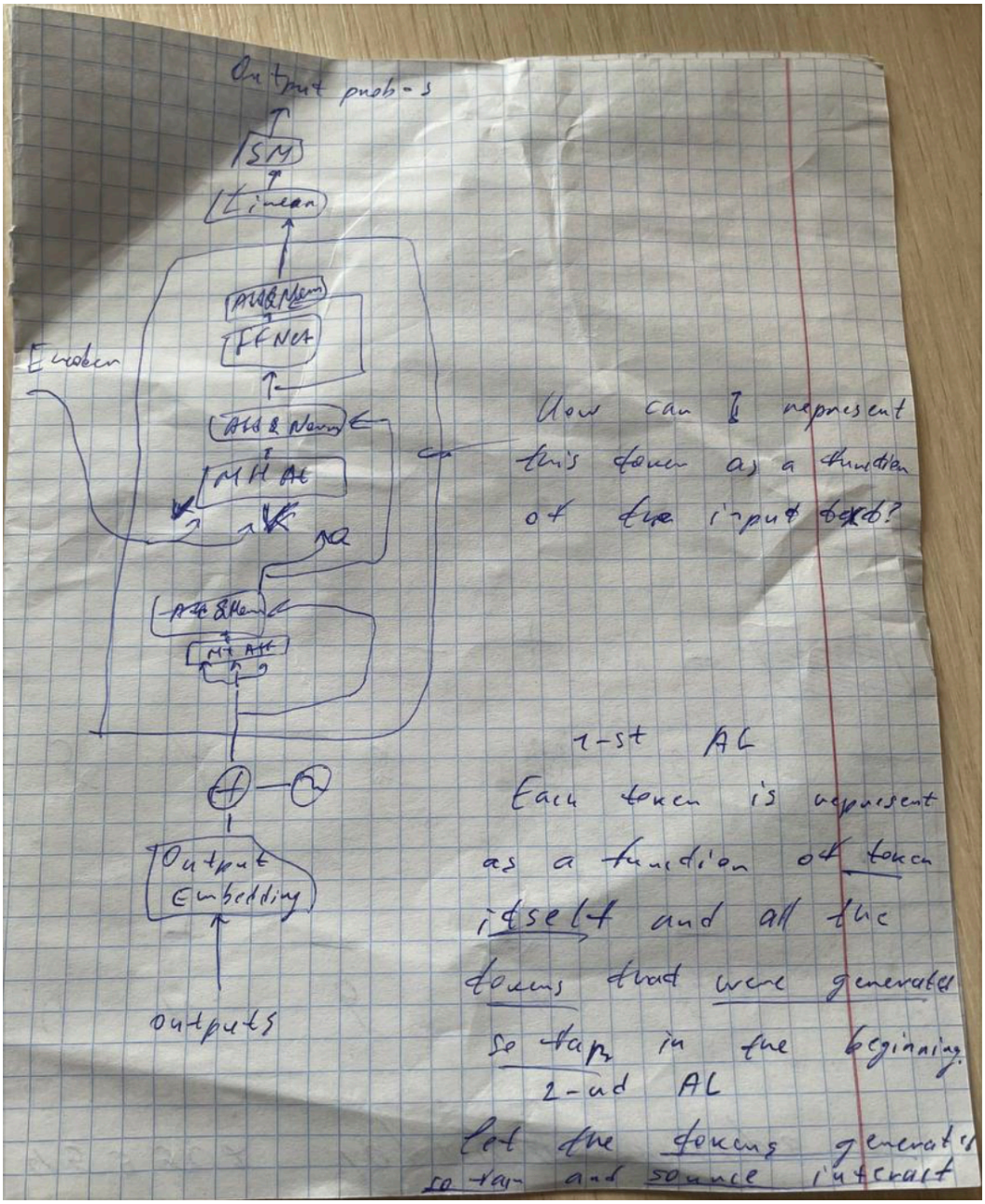
First MH Attention layer Intuition behind

Each token is represented as a function of token itself and all the other tokens generated before

Second MH Attention layer Intuition behind

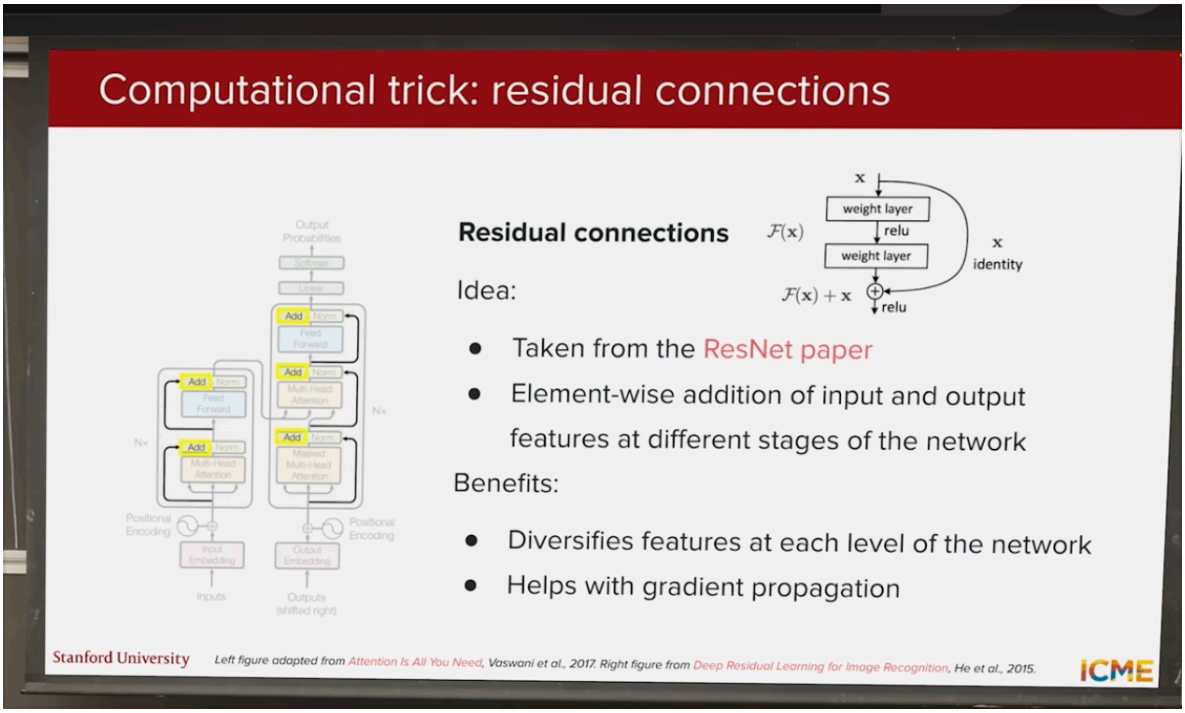
Token is represented as a function of the input text
K and V comes from Encoder part, Q comes from decoder

Output is given as next token using softmax

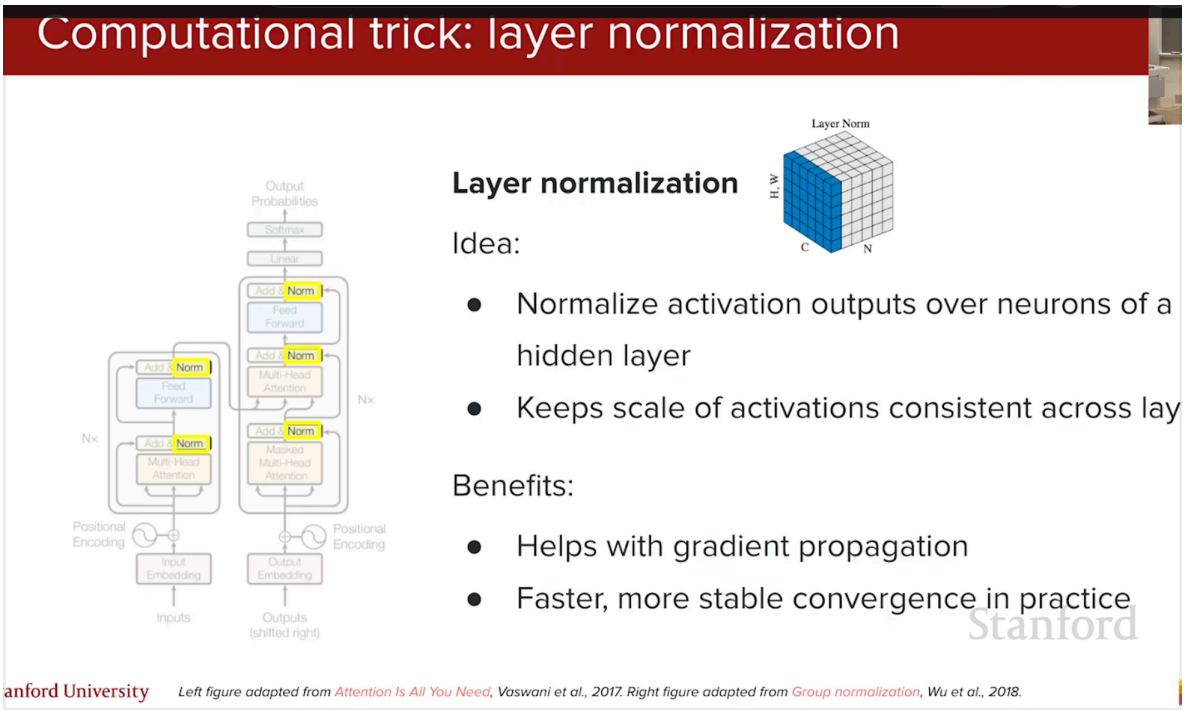


For machine translation you place transformer in ***autoregressive way***, the output of transformer decoder will go to its input and so on until it generates the EOS token

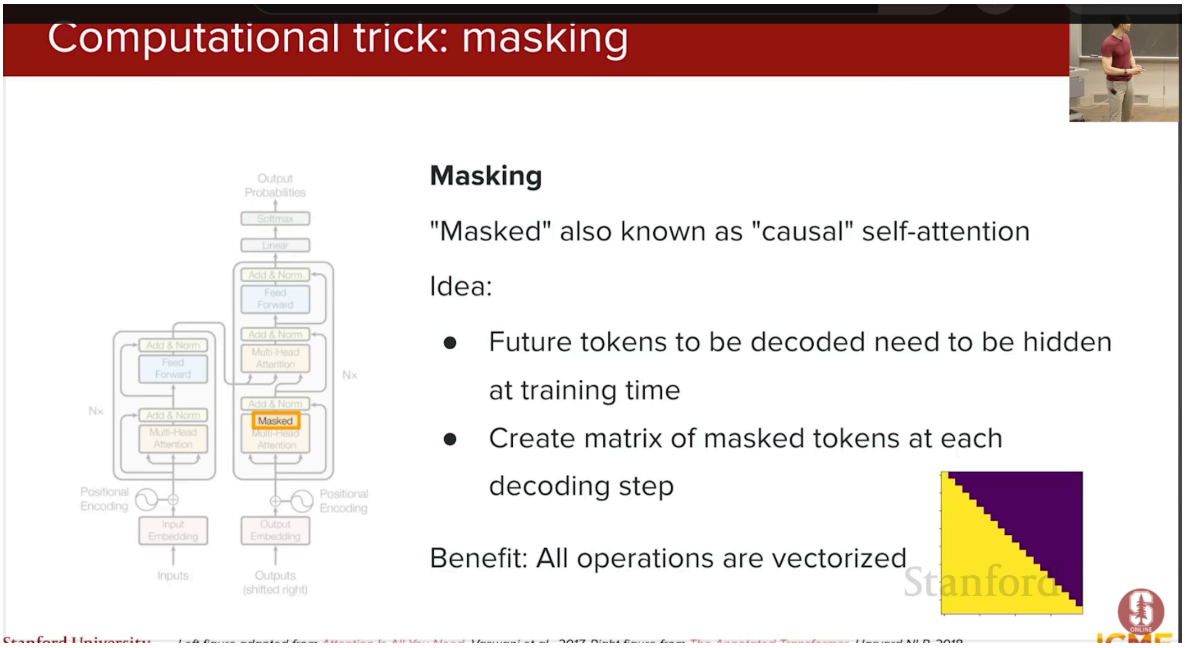
Computational tricks



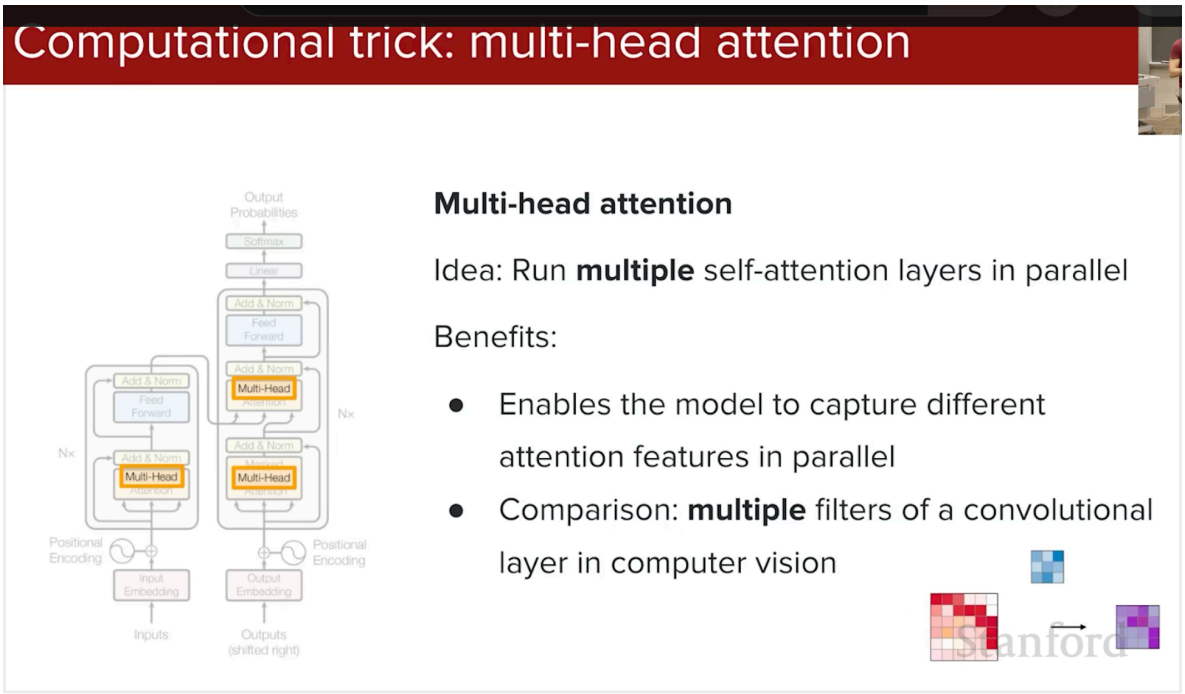
Residual connections allows your model to pass features directly instead of passing their modified version by some sublayers, helps with backpropagation



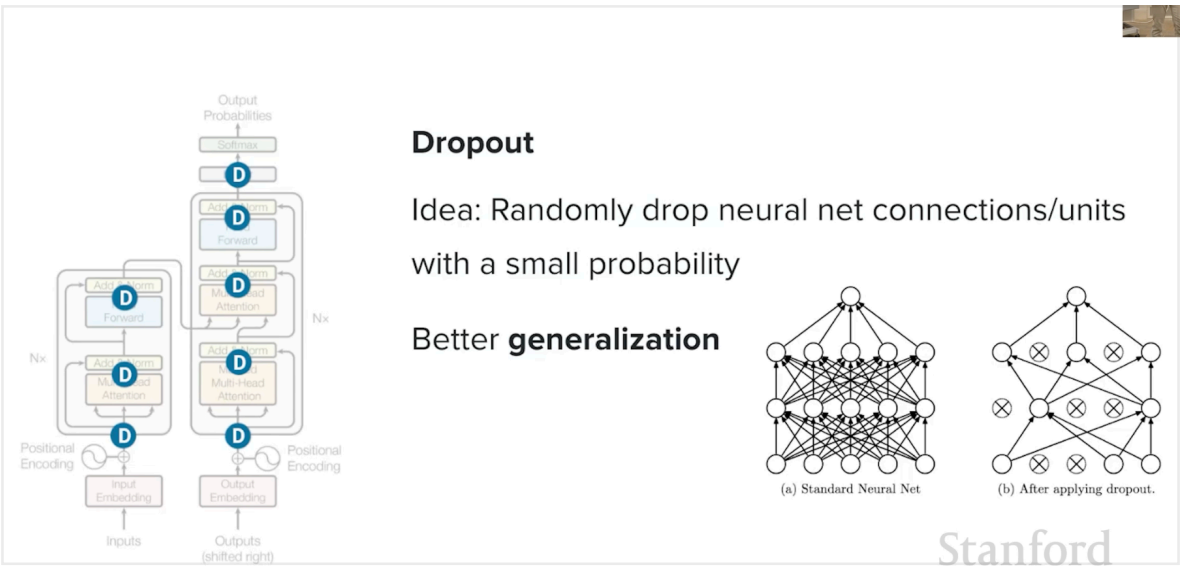
Helps with convergence



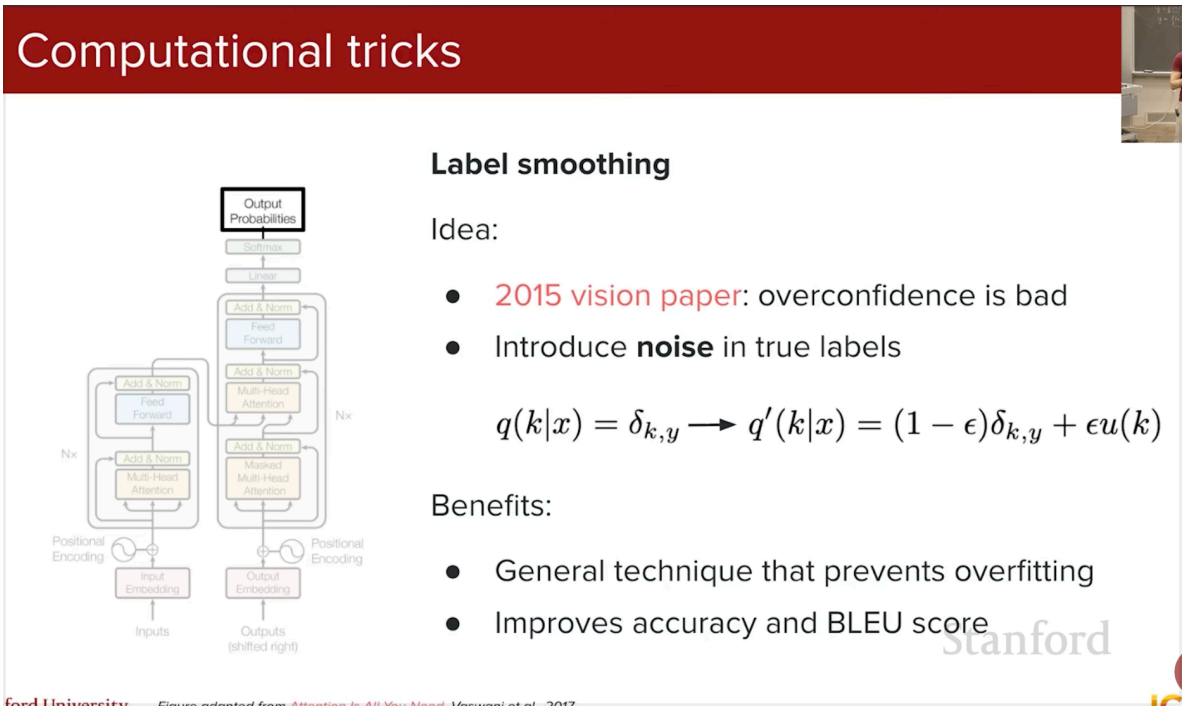
Masking trick is used to force your tokens interact with only decoded tokens, and not yet decoded ones



Analogy with different filters in convolutional neural nets here
Having different attention heads here is a way for your model to have different ways to learn the same thing, through different projection matrices, typically you have 8 such heads



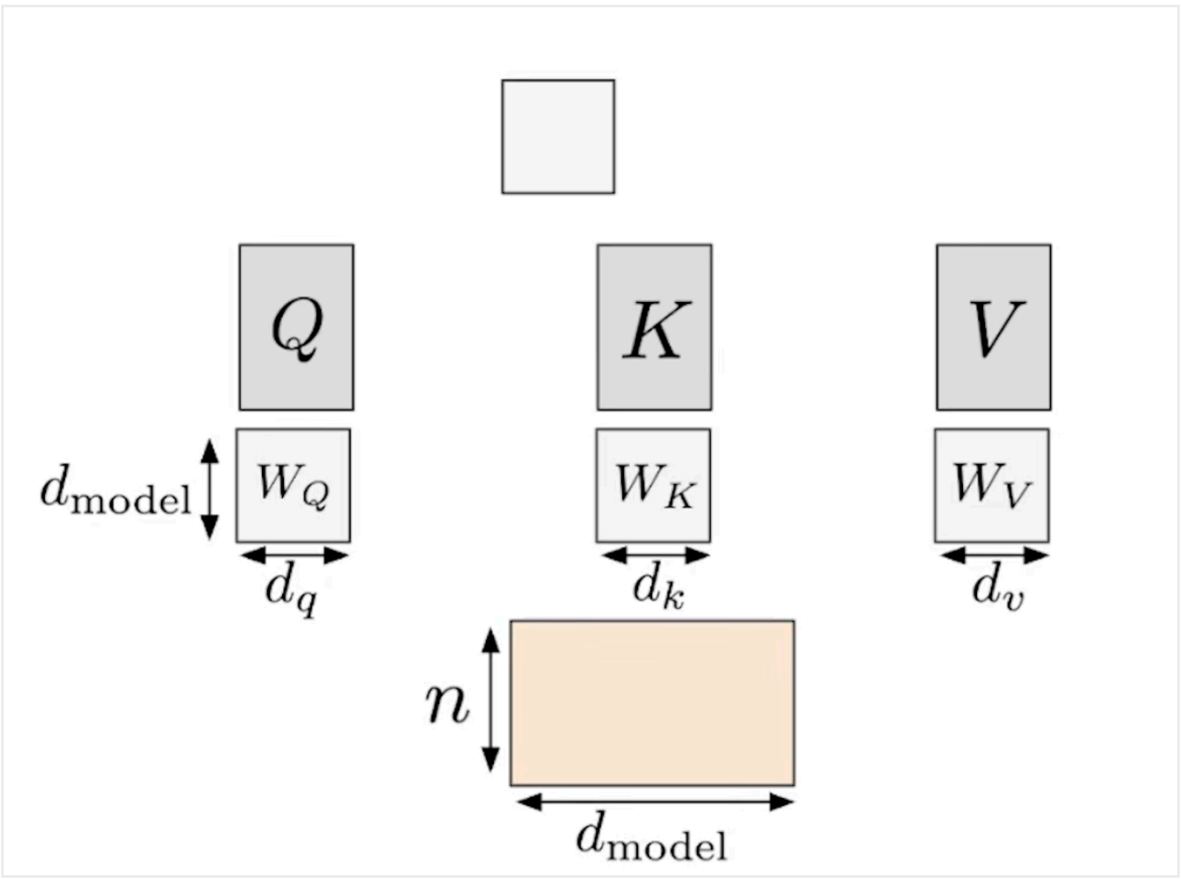
Also in transformer there is applied dropout, not rely too much on some features, that's why it forces your model to generalize better and give better predictions

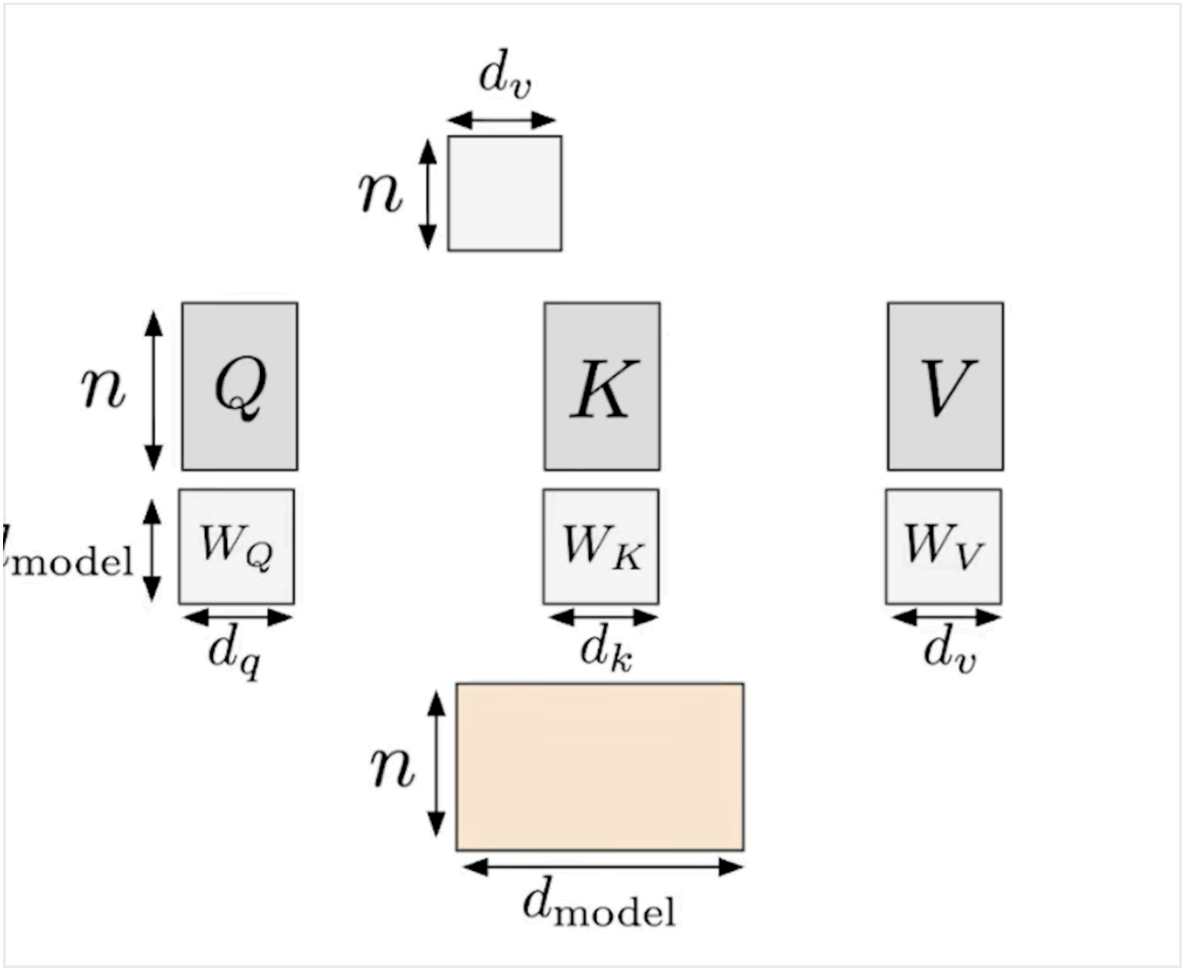


Label smoothing means not giving 100% for some word, improves score like BLEU score

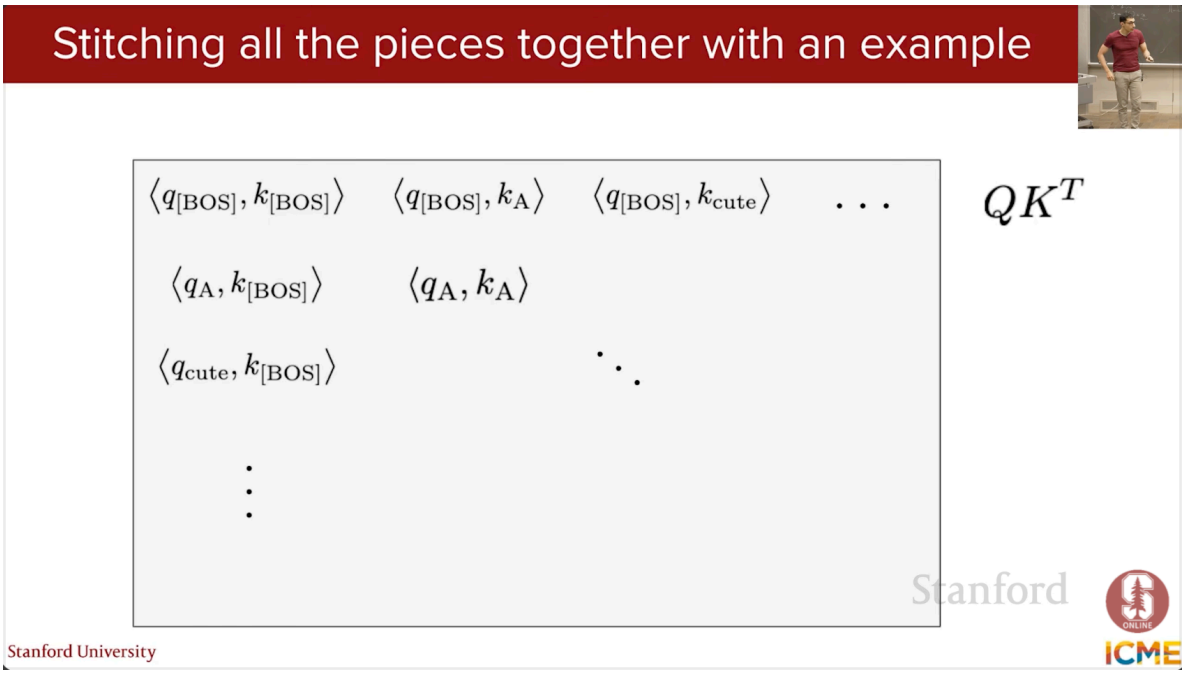
6 END_TO_END EXAMPLE

Watch lecture example, here is a little shot about dimensions and notation
D_model - embedding size
D_q = D_k because Q * K.T

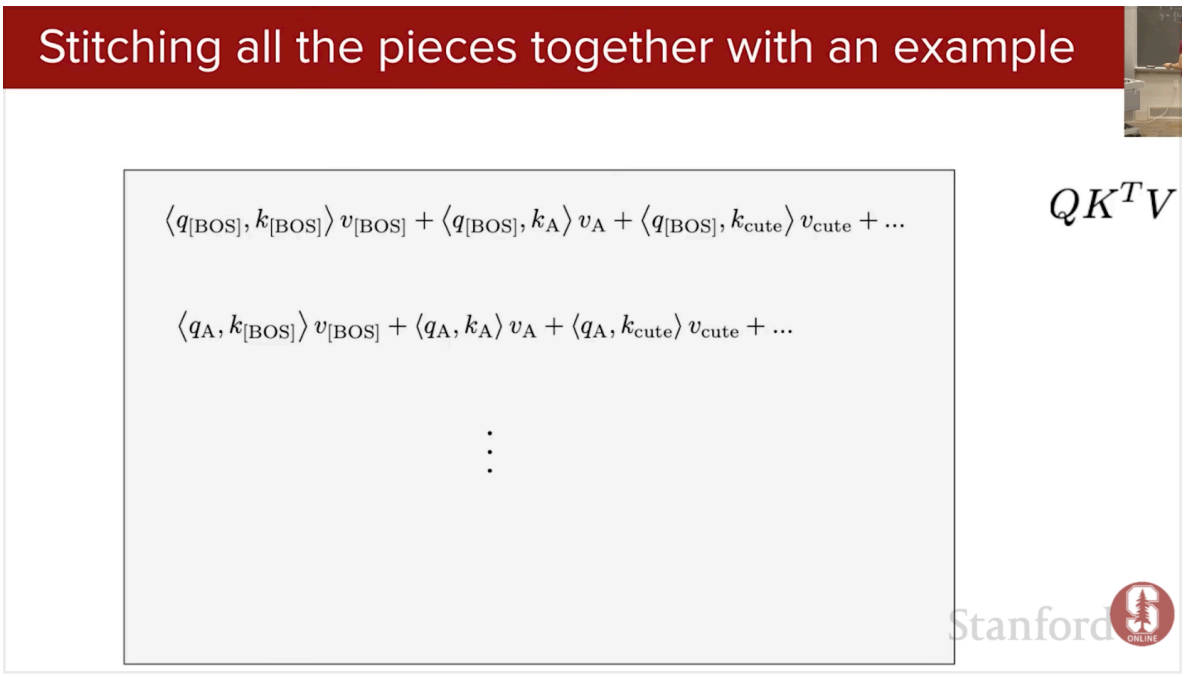

Q
K
V
d_{model}
W_Q
W_K
W_V
d_q
d_k
d_v
n
d_{model}



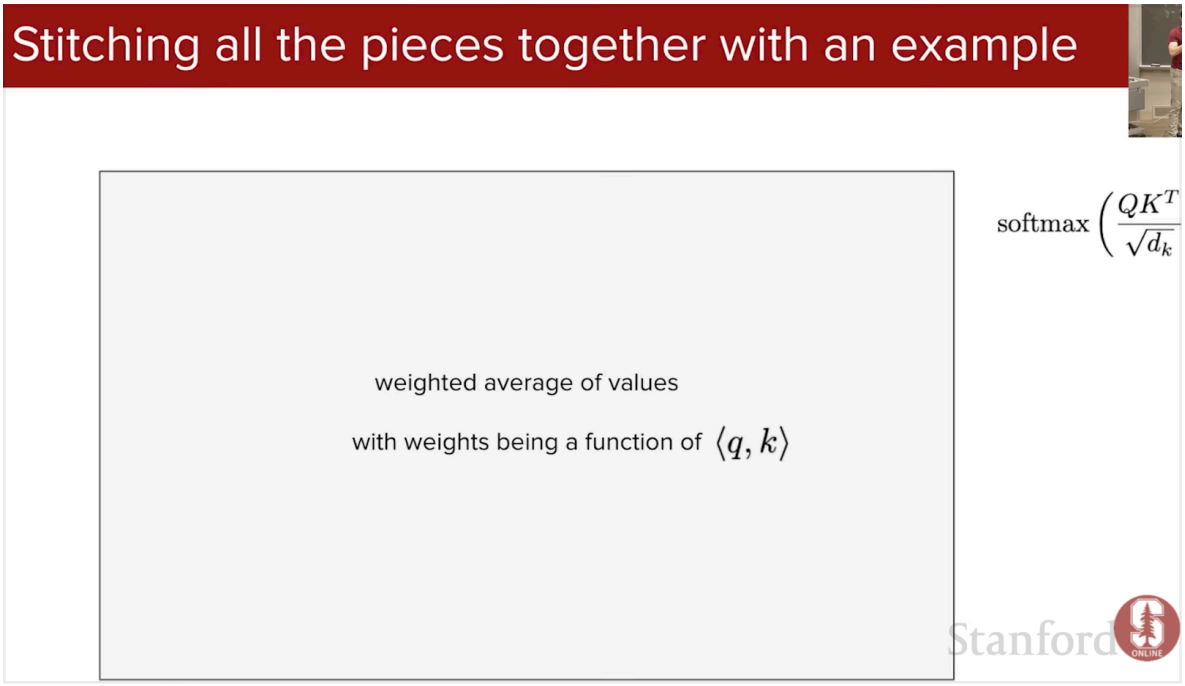
Similarly between given query and given key
Size is (n,n)



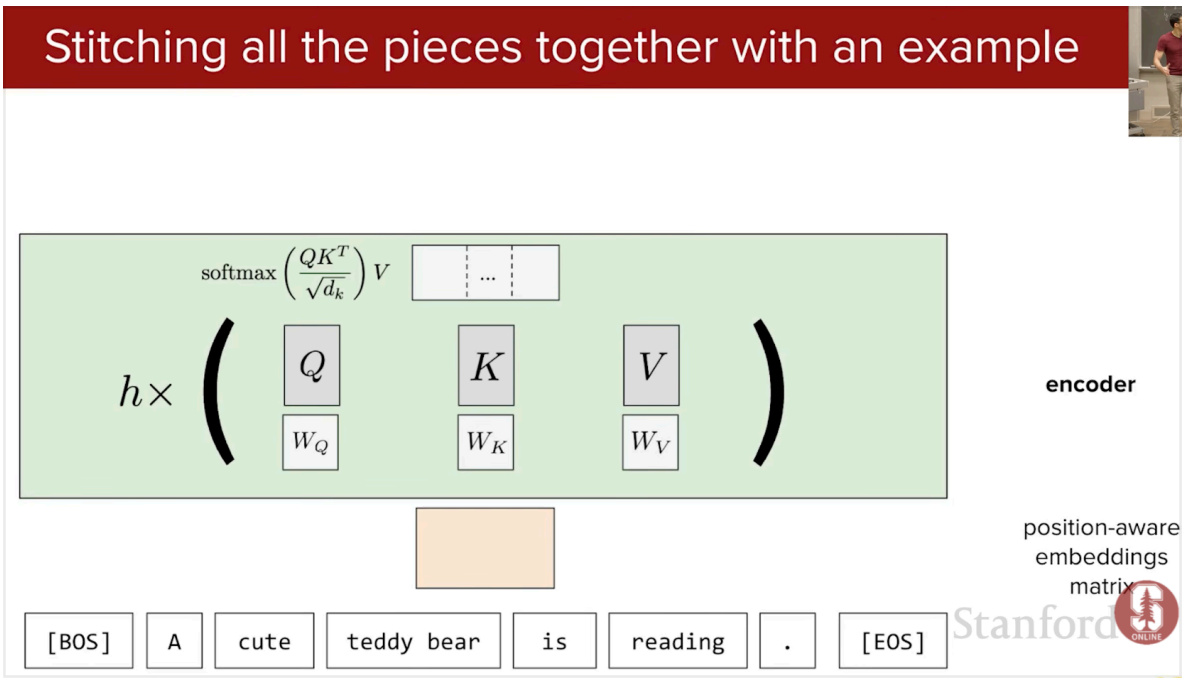
Lets try to multiply by V, it won't be right because we wanted to get some probability distribution first and then use it to weight values



Do like this



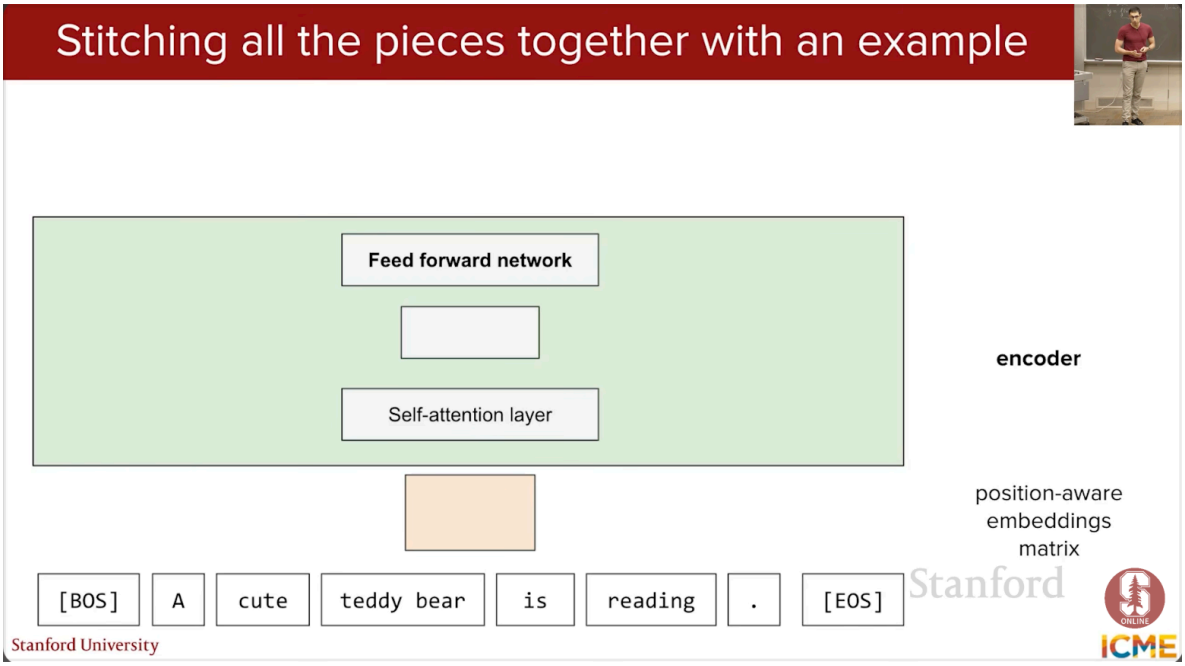
Process of learning Q,K, V is multi-head that means it is done multiple H times in parallel
At the end concatenate it together



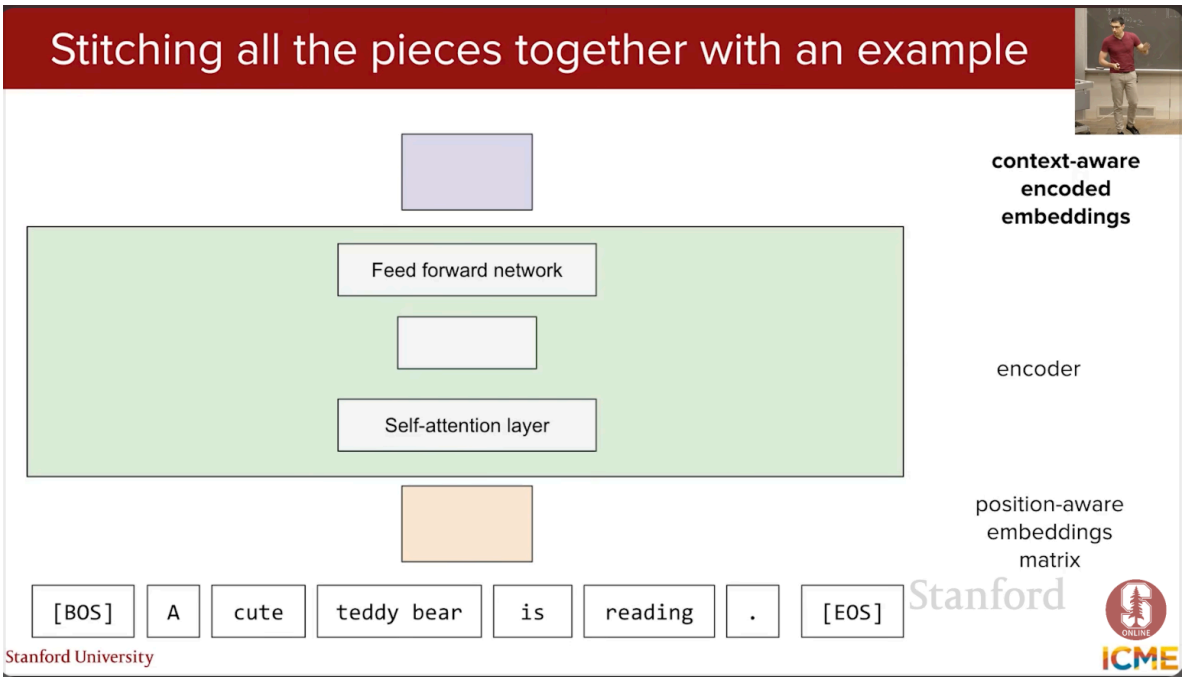
And the last projection matrix W_o that will turn to dimension (H, d_v) back into d_model



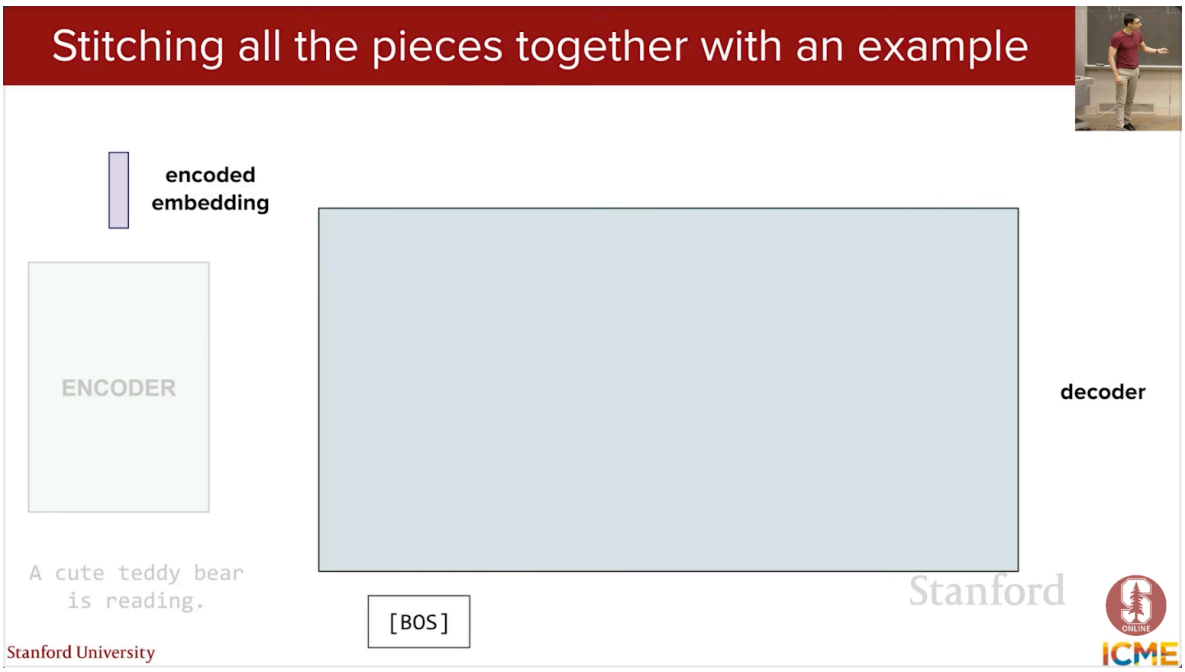
Then feed forward network to learn complex representations and then dimension will be bigger than d_model (original size of embeddings)



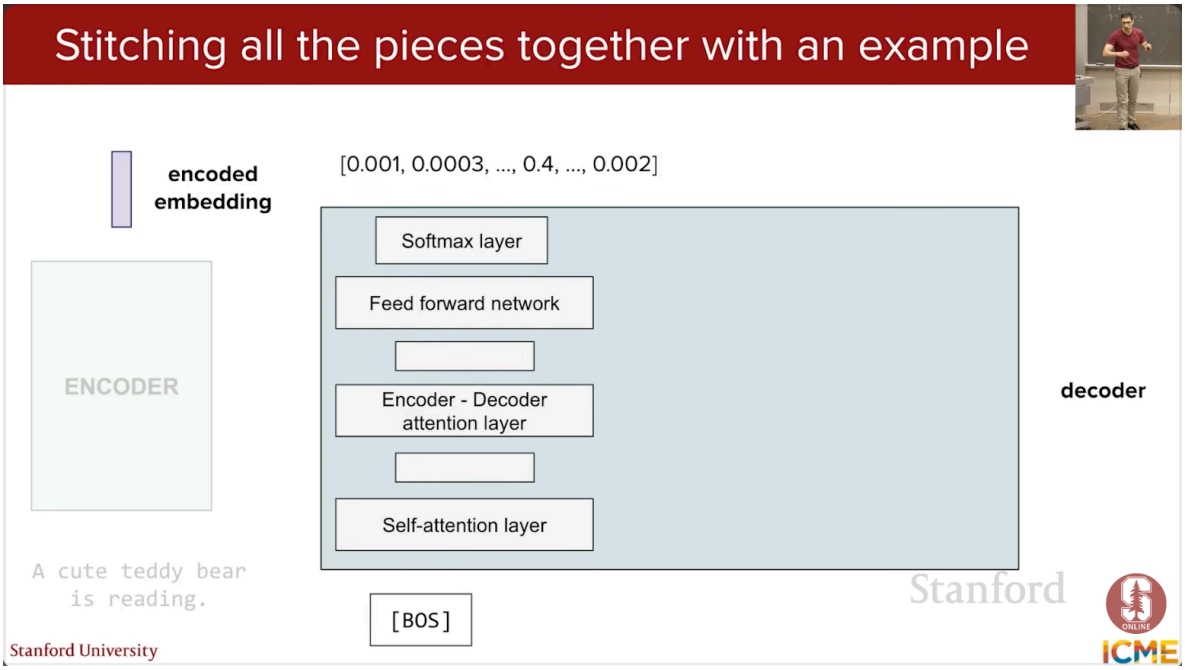
At the end for each token you have it's representation in context-aware way



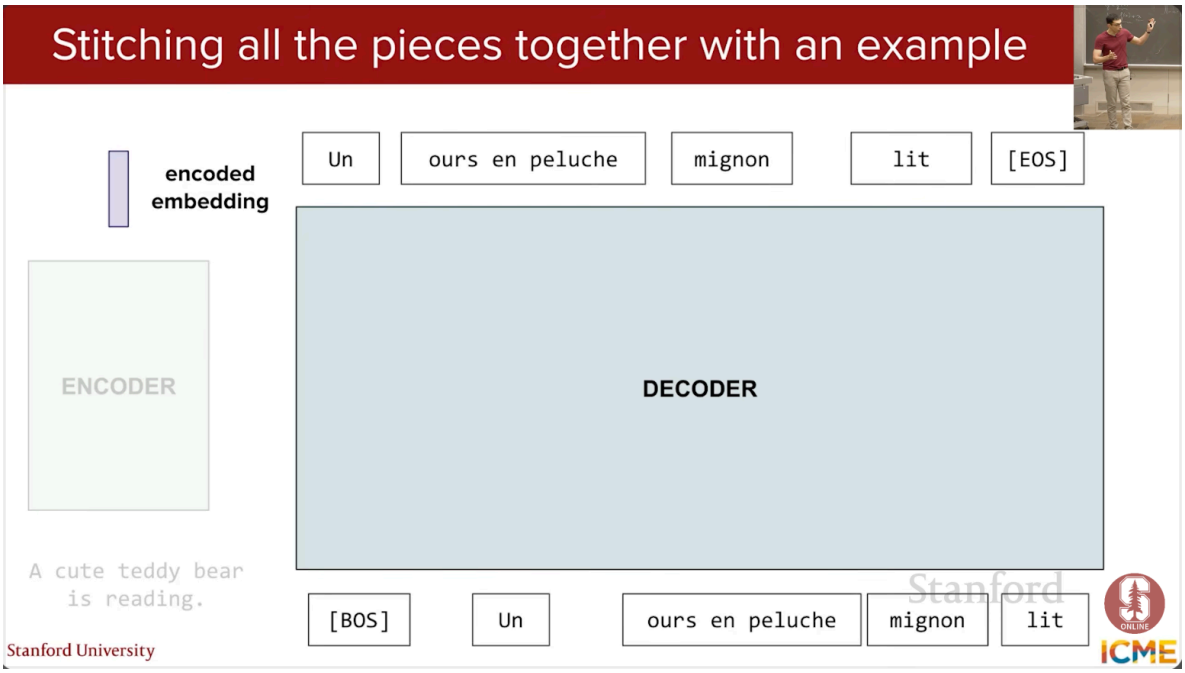
Decoder part
To start translation you need to force it packing BOS token, model looks up at table, gets initial embedding for it as well as positional one and passes it to decoder



After getting softmax probability distributions there are some heuristic techniques that help us to figure out what word to choose, but it will be discussed in next lecture (probably BEAM search or something)



Repeat the process in autoregressive way



Quality of the data for this whole architecture is another layer of complexity
Most of the learning happens with the help of attention and feed forward layer

About masking: prevents making connections that you shouldn't know for now